AI with Rav

Learn AI the way you actually think.

DAY 6 of 30

PYTHON

Booleans & Comparisons — yes/no questions

WHAT YOU'LL LEARN TODAY

- A Boolean is a switch (True/False)
- Comparisons that answer yes/no
- The == vs = trap
- Combining questions with and / or / not
- The simple truth table

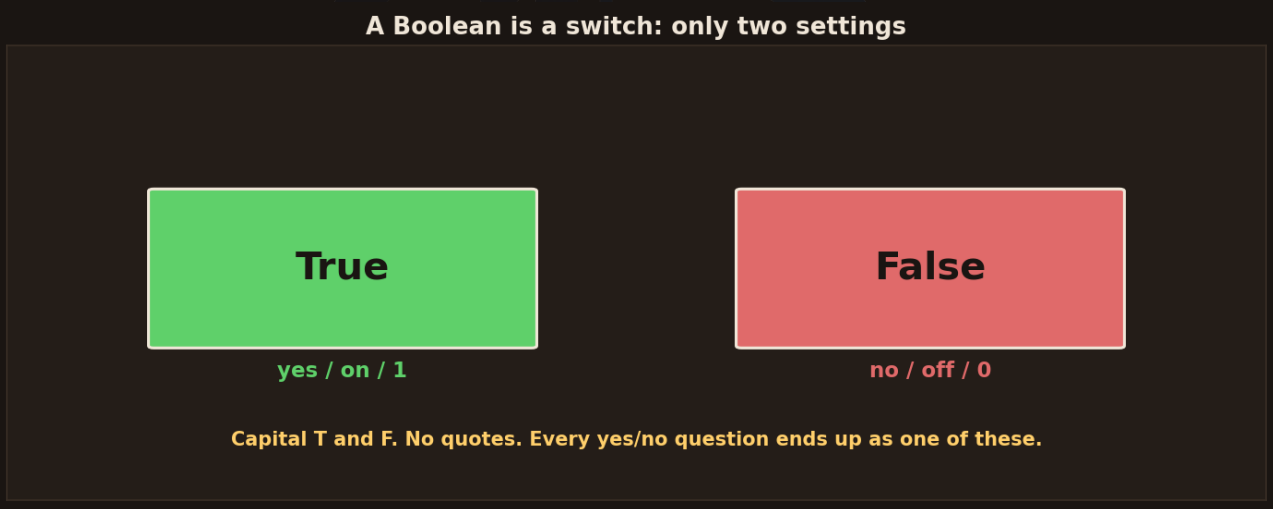
Booleans & Comparisons — yes/no questions

> In One Line: Every decision a program makes starts with a yes/no question — "is the age over 18?", "did the password match?". The answer is always True or False (a Boolean). Today you learn to ASK those questions — the foundation of all decision-making, which we use tomorrow with if.

Start here: computers decide by asking yes/no

A computer looks smart, but under the hood every decision is just a pile of **yes/no questions.** "Is the user logged in?" "Is the cart empty?" "Is the number even?" Each one has exactly two answers — yes or no, `True` or `False`. That answer is called a **Boolean.** Master asking these questions and you can make your program *decide* things.

A Boolean is a switch



A Boolean is a switch with only two settings: True (yes / on) shown in green, or False (no / off) shown in red. Capital T and F, no quotes. Every yes/no question ends up as one of these two.

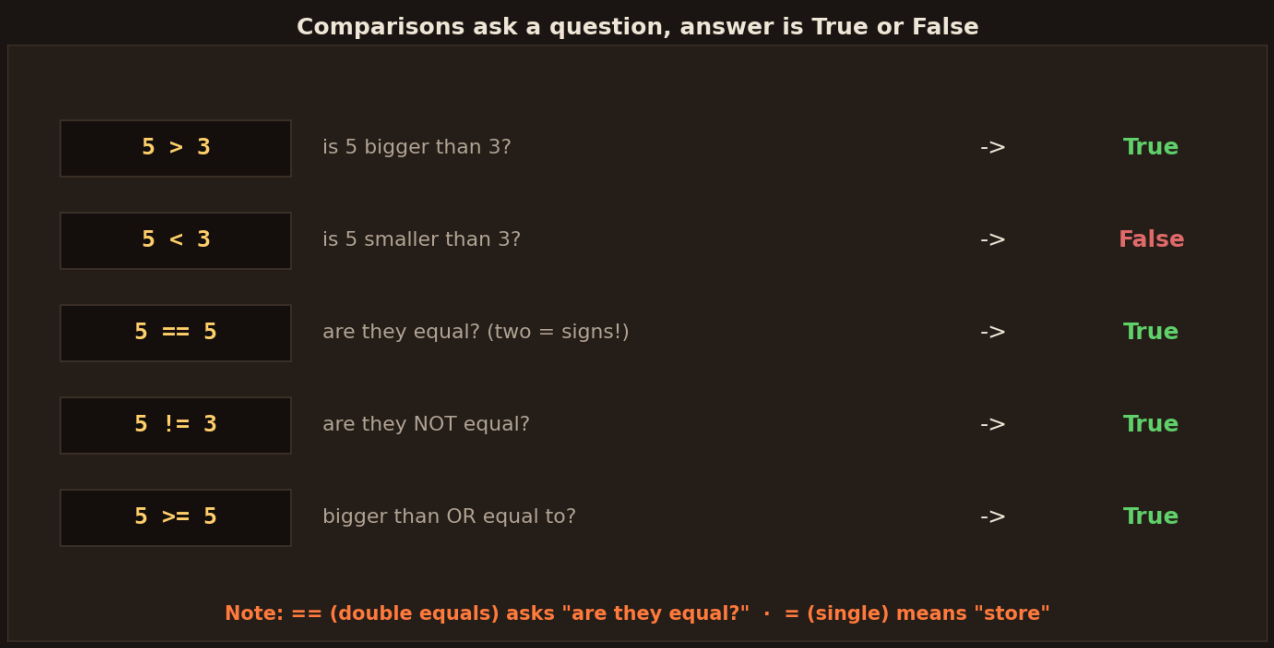
You met `bool` briefly on Day 2. It's the simplest type — a switch with only **two** settings:

- `True` → yes / on.
- `False` → no / off.

```
is_raining = True
is_weekend = False
print(is_raining)     # -> True
```


> Note the capital `T` and `F`, and **no quotes** — `True` is the Boolean value, but `"True"` (with quotes) would just be text. **(If you already code:** same idea as boolean in Java, but capitalised: `True`/`False`, not `true`/`false`.**)** These two values are what every question below boils down to.

Comparisons — asking a question



Comparison operators and their answers: 5 > 3 gives True, 5 < 3 gives False, 5 == 5 gives True, 5 != 3 gives True, 5 >= 5 gives True. Note == (double equals) asks are-they-equal; single = means store.

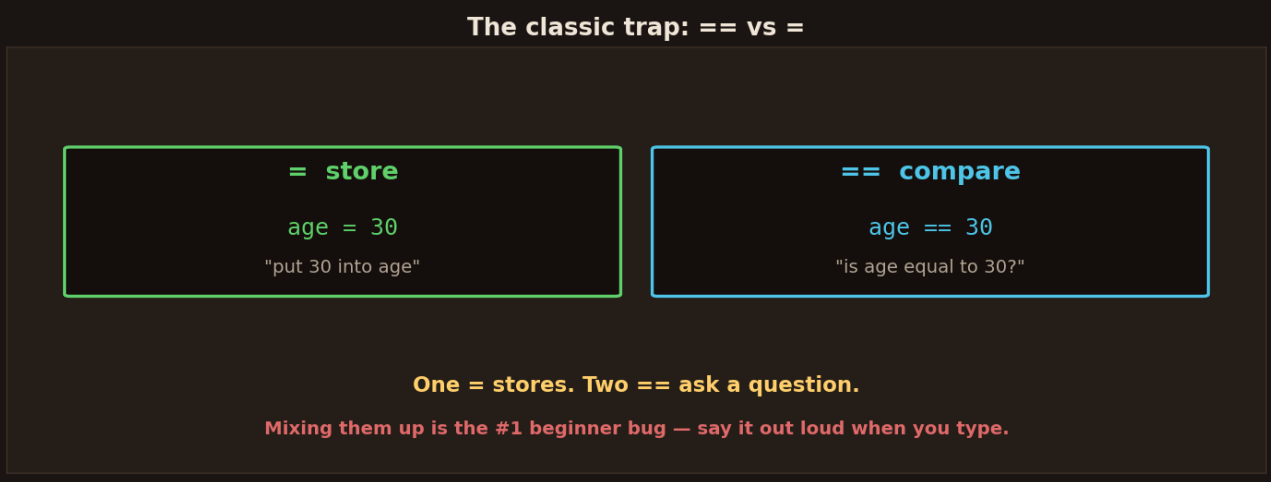
You don't type `True`/`False` by hand much — usually Python works them out by **comparing** two things. Each comparison asks a question and answers `True` or `False`:

- `>` greater than, `<` less than — `5 > 3` → `True`.
- `>=` greater-or-equal, `<=` less-or-equal — `5 >= 5` → `True`.
- `==` **equal to** (two equals signs!) — `5 == 5` → `True`.
- `!=` **not equal to** — `5 != 3` → `True`.

```
print(5 > 3)       # -> True
print(5 == 5)      # -> True   (are they equal?)
print(5 != 3)      # -> True   (are they NOT equal?)
age = 20
print(age >= 18)   # -> True   (is age at least 18?)
```


> A comparison is a **question the computer answers.** `age >= 18` literally means "is age at least 18?" and comes back `True` or `False`. This is how a program checks conditions — is the price too high, did the answer match, is the list empty. Everything decisions are built on.

The classic trap: == vs =



One equals sign = means STORE (age = 30 puts 30 into age). Two equals signs == means COMPARE (age == 30 asks is age equal to 30). Mixing them up is the number one beginner bug.

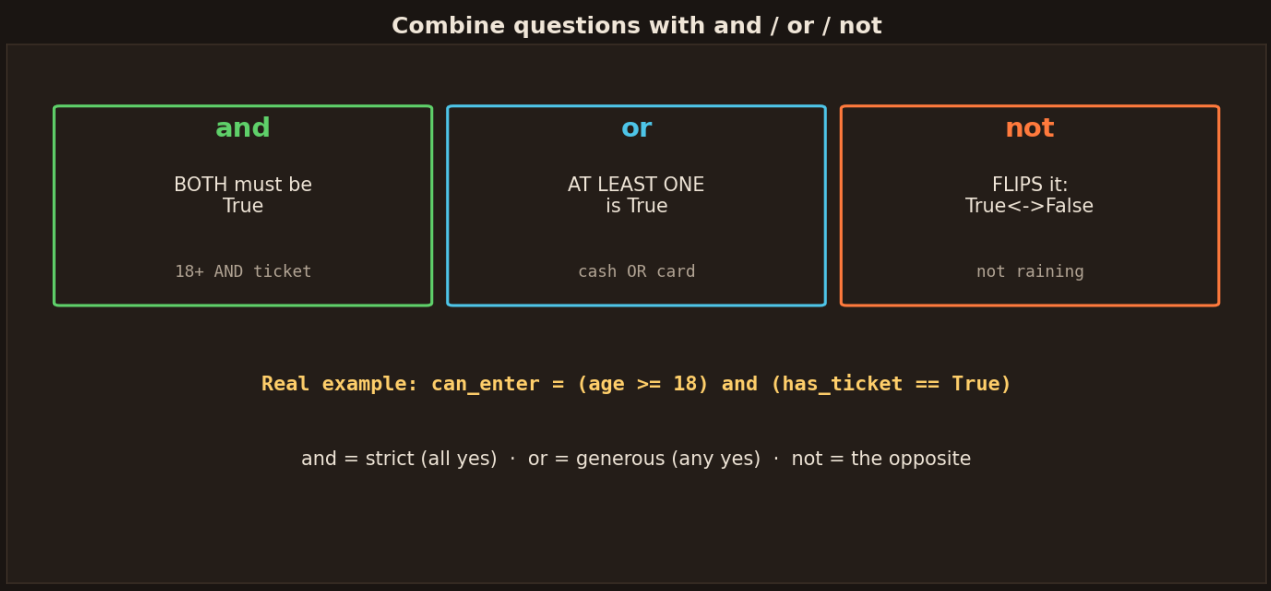
This one bites *every* beginner, so let's nail it now:

- `=` (one equals) → **store** a value: `age = 30` means "put 30 into age."
- `==` (two equals) → **compare**: `age == 30` asks "is age equal to 30?"

```
age = 30           # STORE: put 30 into age
print(age == 30)   # COMPARE: is age equal to 30?  -> True
print(age == 40)   # -> False
```

> Say it out loud as you type: **one equals = "make it so", two equals = "is it so?"** Using `=` when you meant `==` (or the other way) is the single most common early bug. When you write a question, you almost always want `==`.

Combining questions — and / or / not



Three ways to combine questions: and (BOTH must be True), or (at least one is True), not (flips True to False and back). Example: can_enter = (age >= 18) and (has_ticket == True). and is strict, or is generous, not is the opposite.

Real decisions often need *more than one* condition. Combine them with three plain-English words:

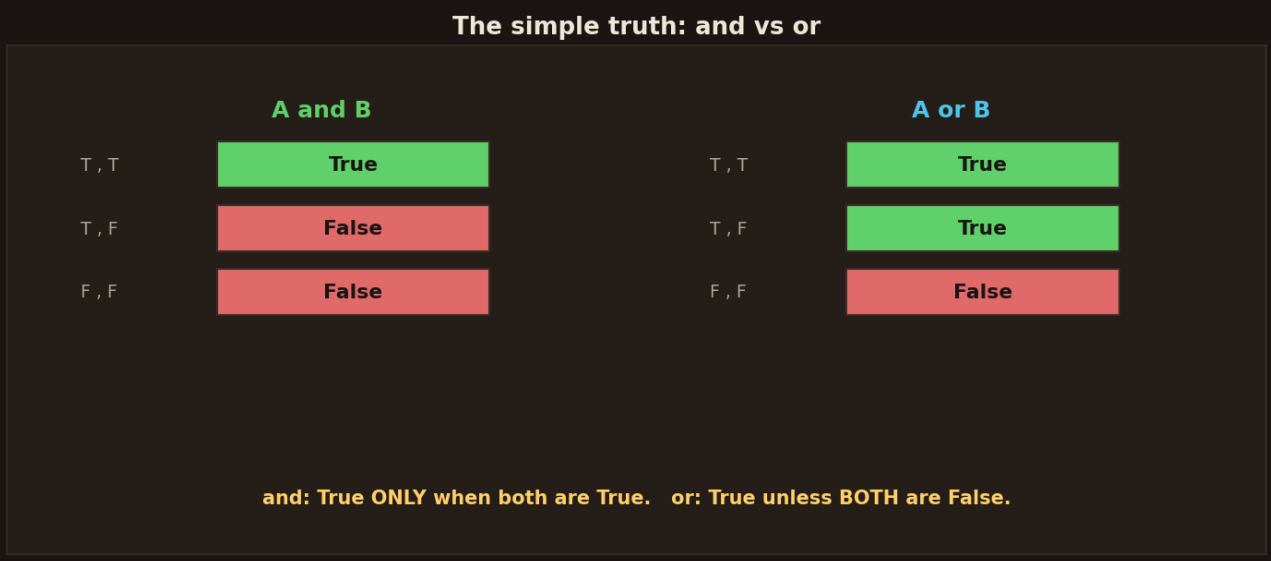
- `and` → **True only if BOTH** are true. "18+ **and** has a ticket."
- `or` → **True if AT LEAST ONE** is true. "pays by cash **or** card."
- `not` → **flips** it. `not True` is `False`.

```
age = 20
has_ticket = True
can_enter = (age >= 18) and (has_ticket == True)
print(can_enter)      # -> True   (both are true)

print(not True)       # -> False  (flips it)
```

> Read them like English: **and** is strict (everything must be yes), **or** is generous (any one yes is enough), **not** is the opposite. The brackets `( )` aren't required but make your intention clear — group each question so anyone reading knows exactly what's being asked.

The simple truth



Truth table: for A and B, the result is True only when both are True (T,T), otherwise False. For A or B, the result is True unless both are False (F,F). Green cells are True, red cells are False.

If you ever forget how `and`/`or` behave, remember these two lines:

- `and` → gives `True` **only when both** sides are `True`.
- `or` → gives `True` **unless both** sides are `False`.

> That's the whole rule. **and** is the demanding boss — everyone must say yes. **or** is the easygoing one — one yes is enough. You'll lean on these constantly: "is the form filled AND the box ticked?", "is it a weekend OR a holiday?". Next we'll finally *use* these answers to make the program branch.

Recap — the 20-second version

- A **Boolean** is a switch: `True` or `False` (capital, no quotes).
- **Comparisons** answer yes/no: `>` `<` `>=` `<=` `==` (equal) `!=` (not equal).
- `=` **stores**, `==` **compares** — the #1 beginner trap.
- Combine questions with `and` (both), `or` (either), `not` (flip).
- `and` = True only if both; `or` = True unless both are False.

> Next up — Video 7: If / Elif / Else. Now that you can ask yes/no questions, let's make the program ACT on the answer — do one thing if True, another if False. It's the road that forks, and it's where your code starts making real decisions. See you tomorrow.